

# Overview

---

- Operating systems provide an environment for execution of programs and services to programs and users
- Can be viewed from several vantage points:
  - **Services** that system provides
  - **Interface** it makes available to users and programmers
  - Components and their **interactions**

# Operating System Services

---

Services that are helpful to user

- **User interface** - Almost all operating systems have a user interface (**UI**).
  - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
- **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
- **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file information, permission management.
- **Communications** – Processes may exchange information, on the same computer or between computers over a network
  - **Communications** may be via **shared memory** or through **message passing** (packets moved by the OS)

# Operating System Services (Cont.)

---

- **Error detection** – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Services that ensure efficient system operation

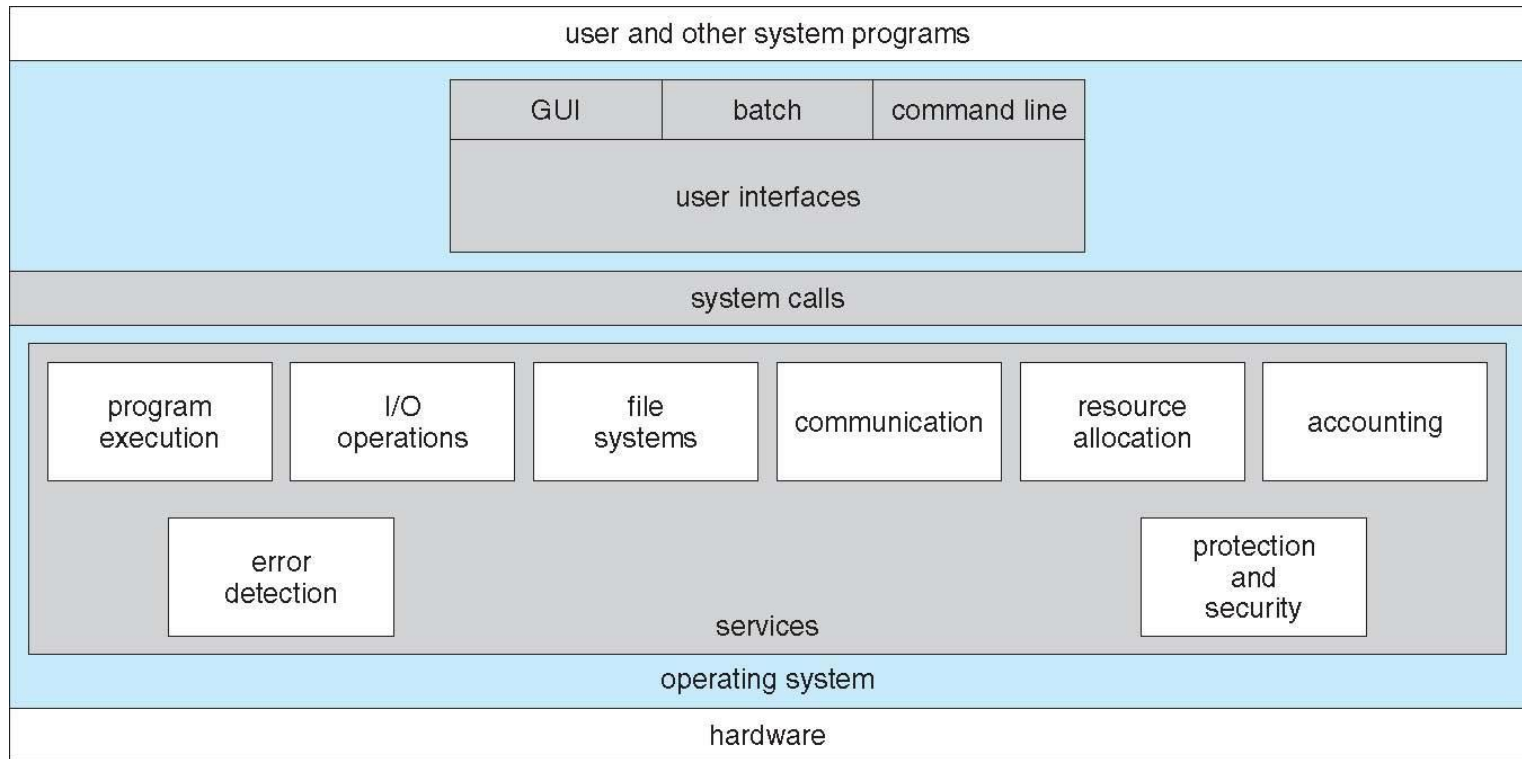
- **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
  - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
- **Accounting** - To keep track of which users use how much and what kinds of computer resources

# Operating System Services (Cont.)

---

- **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
  - **Protection** involves ensuring that all **access to system resources** is controlled
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

# A View of Operating System Services



# User Operating System Interface - CLI

---

CLI or **command interpreter** allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
  - 4 If the latter, adding new features doesn't require shell modification

# User Operating System Interface - GUI

---

- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI “command” shell
  - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

# Touchscreen Interfaces

- Touchscreen devices require new interfaces
  - Mouse not possible or not desired
  - Actions and selection based on gestures
  - Virtual keyboard for text entry
- Voice commands.





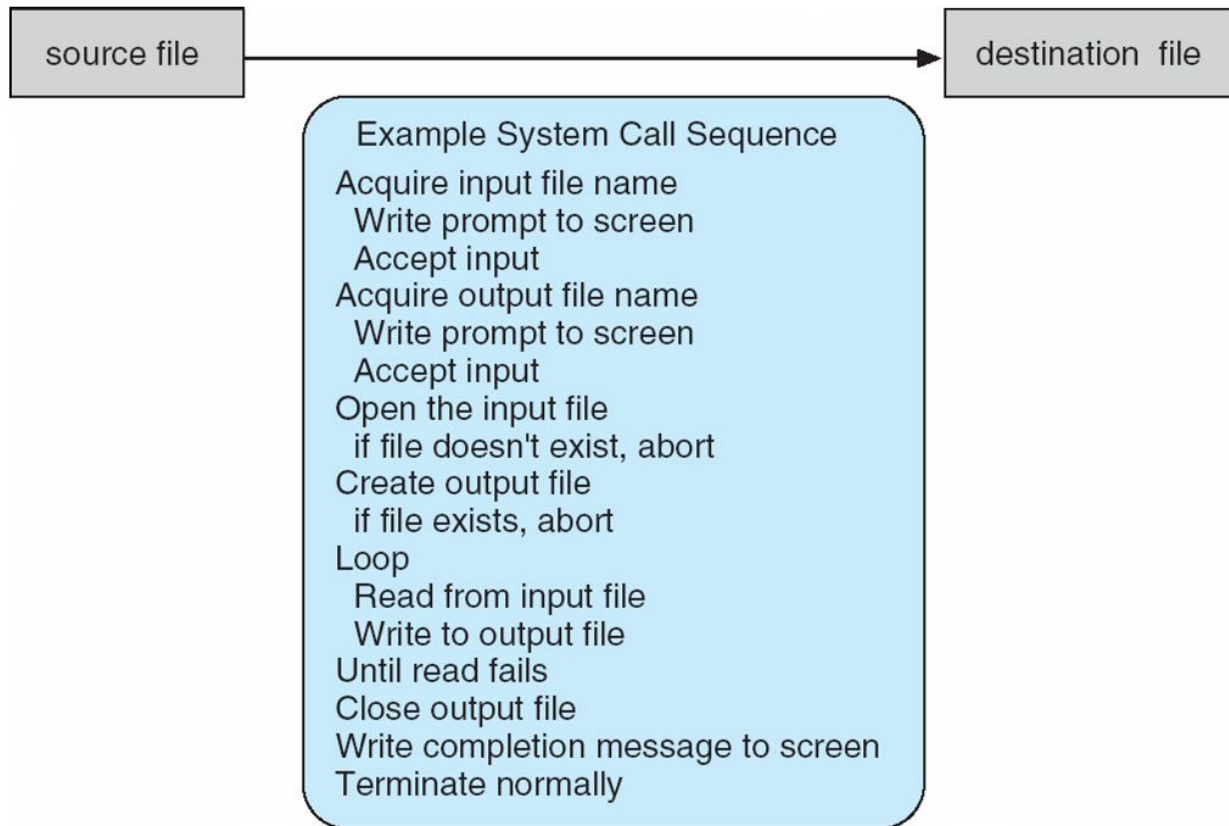
# System Calls

---

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

# Example of System Calls

- System call sequence to copy the contents of one file to another file

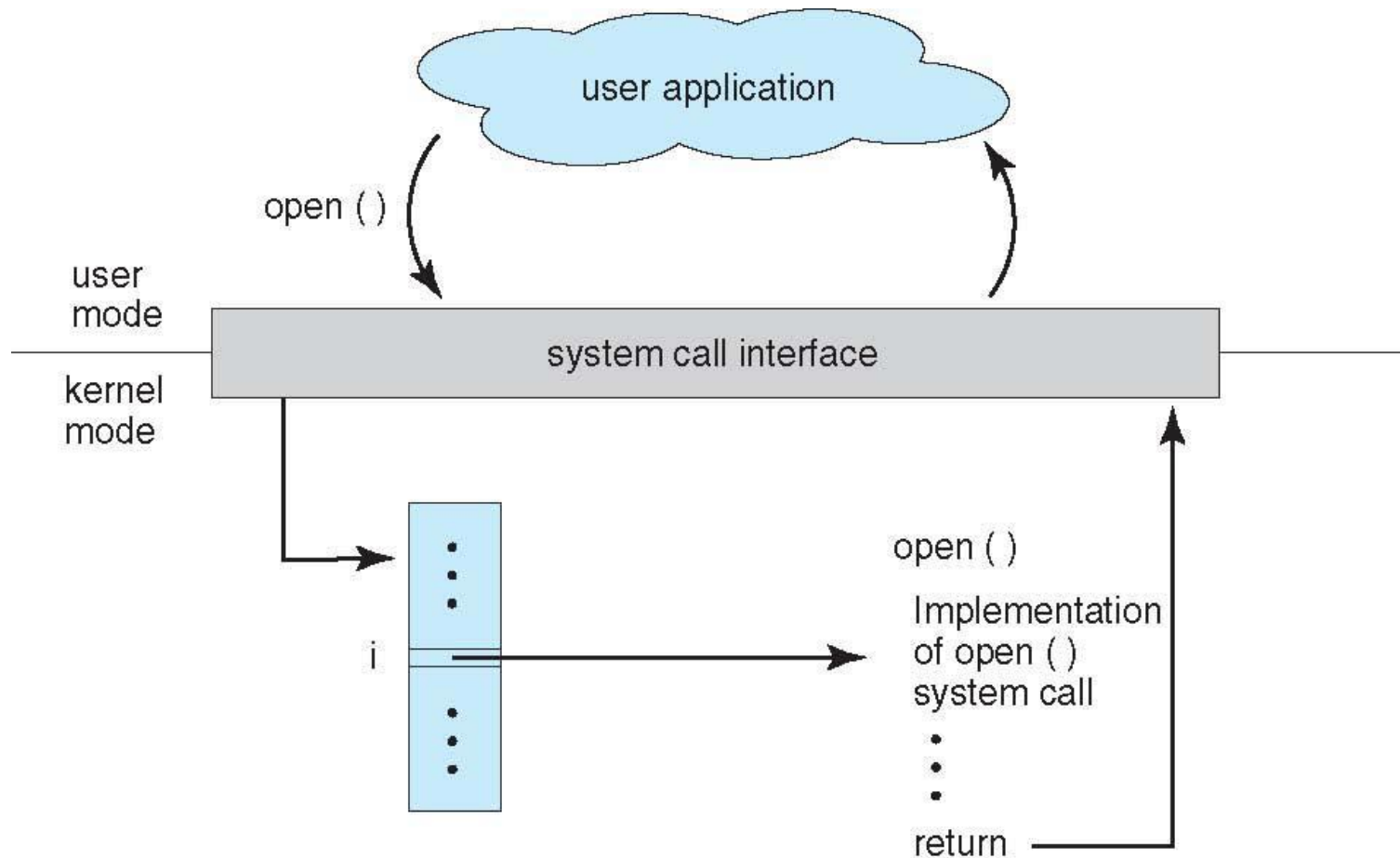


# System Call Implementation

---

- Typically, a number associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
  - 4 Managed by run-time support library (set of functions built into libraries included with compiler)

# API – System Call – OS Relationship



# Example of Standard API

## EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t  read(int fd, void *buf, size_t count)
```

|                 |                  |            |
|-----------------|------------------|------------|
| return<br>value | function<br>name | parameters |
|-----------------|------------------|------------|

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

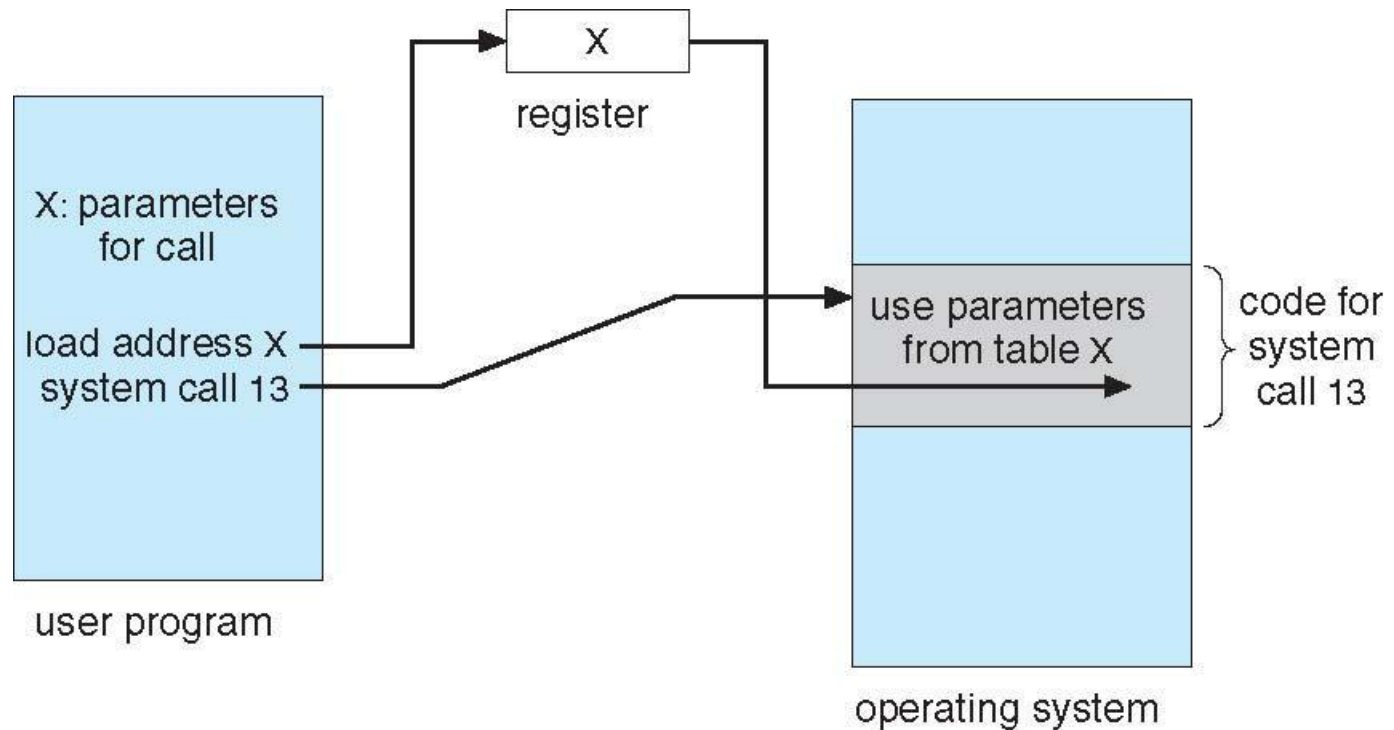
On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

# System Call Parameter Passing

---

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - 4 In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - 4 This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

# Parameter Passing via Table



# Types of System Calls

---

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes



# Types of System Calls

---

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

# Types of System Calls (Cont.)

---

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - 4 From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

# Types of System Calls (Cont.)

---

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

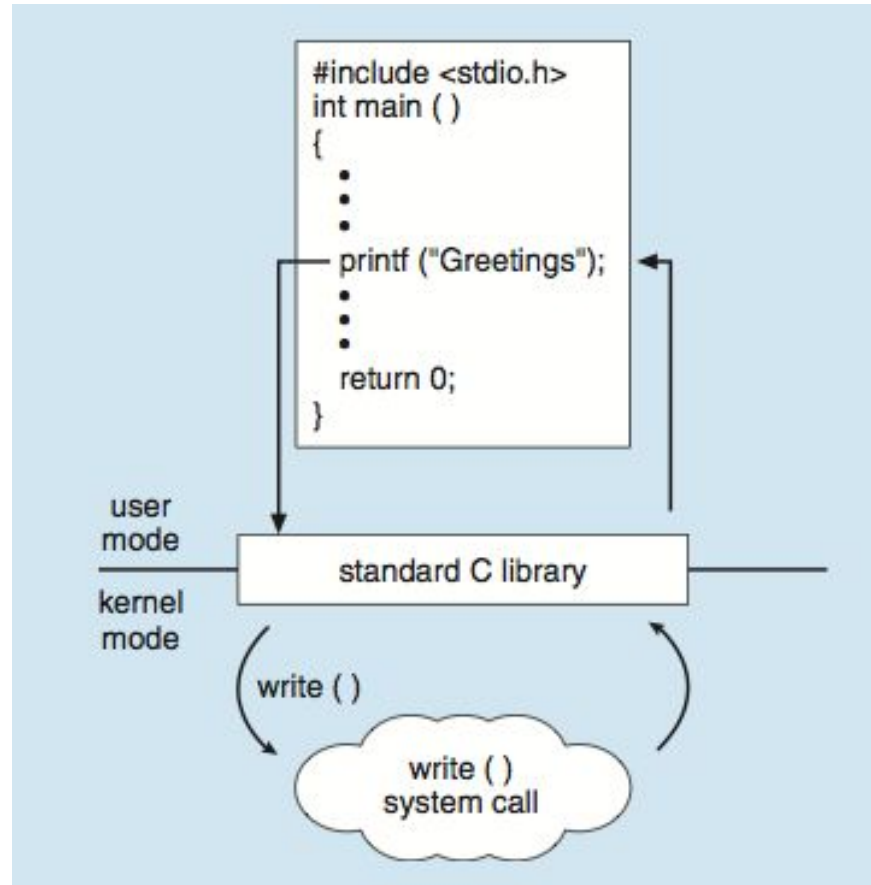
# Examples of Windows and Unix System Calls

---

|                         | Windows                                                                             | Unix                                   |
|-------------------------|-------------------------------------------------------------------------------------|----------------------------------------|
| Process Control         | CreateProcess()<br>ExitProcess()<br>WaitForSingleObject()                           | fork()<br>exit()<br>wait()             |
| File Manipulation       | CreateFile()<br>ReadFile()<br>WriteFile()<br>CloseHandle()                          | open()<br>read()<br>write()<br>close() |
| Device Manipulation     | SetConsoleMode()<br>ReadConsole()<br>WriteConsole()                                 | ioctl()<br>read()<br>write()           |
| Information Maintenance | GetCurrentProcessID()<br>SetTimer()<br>Sleep()                                      | getpid()<br>alarm()<br>sleep()         |
| Communication           | CreatePipe()<br>CreateFileMapping()<br>MapViewOfFile()                              | pipe()<br>shmget()<br>mmap()           |
| Protection              | SetFileSecurity()<br>InitializeSecurityDescriptor()<br>SetSecurityDescriptorGroup() | chmod()<br>umask()<br>chown()          |

# Standard C Library Example

- C program invoking printf() library call, which calls write() system call



# System Programs

---

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information sometimes stored in a File modification
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls

# Operating System Design and Implementation

---

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- **User** goals and **System** goals
  - User Goals: Operating system should be convenient to use, easy to learn, reliable, safe, and fast, responsive
  - Operating system should be easy to design, implement, and maintain, as well as flexible (not one size fits all), reliable, agile (adopting to change in application needs), error-free, and efficient
- **Performance, Protection, Extensibility**

# Operating System Structure

---

- General-purpose OS is very large program
- Various ways to structure ones
  - Monolithic Structure
  - Microkernel Structure
  - Layered
  - Exokernel



# Monolithic Structure

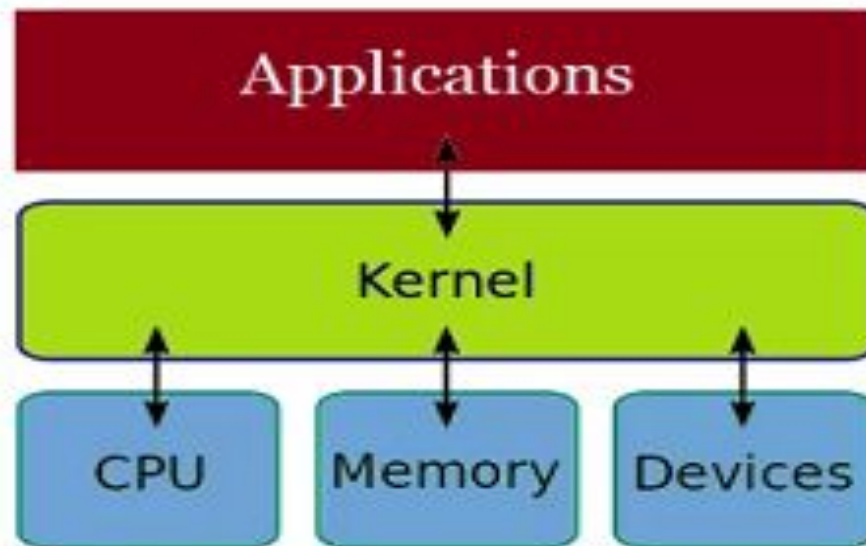
---

- Entire Operating System runs as a single program in kernel mode
- Collection of **procedures** (services, device drivers, file system, access to peripheral devices) linked together to form a single larger program
  - Each procedure is free to call any other procedure
    - 4 Makes system extremely efficient
  - Leads to unwieldy and difficult to understand system
    - 4 Crash in any system take down the entire operating system, e.g. bug in audio driver can be lethal
  - No opportunity for customization
- No information hiding within kernel, every system visible to every other system
- Applications on top of kernel, in their protected address space
- Operating system in its own hardware address space (**OS protected from applications**)

# Monolithic Structure

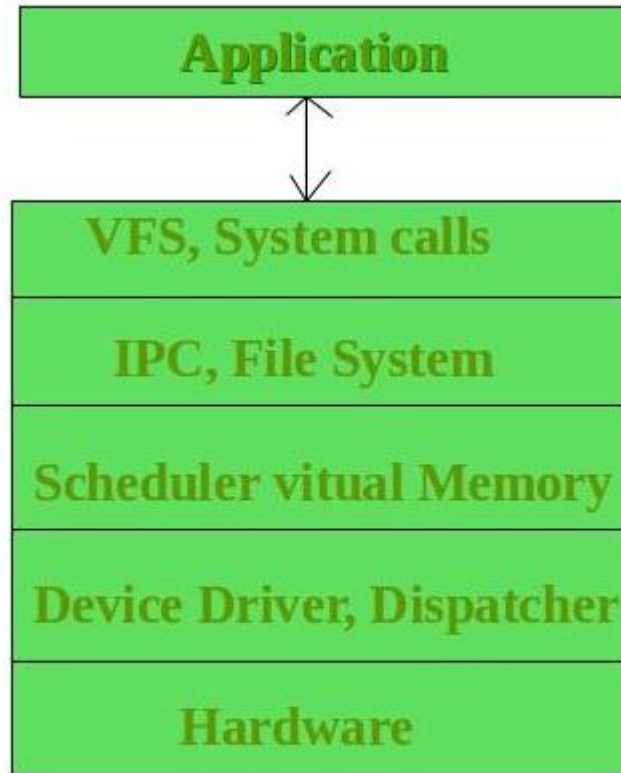
---

## Kernel In OS



# Monolithic Structure

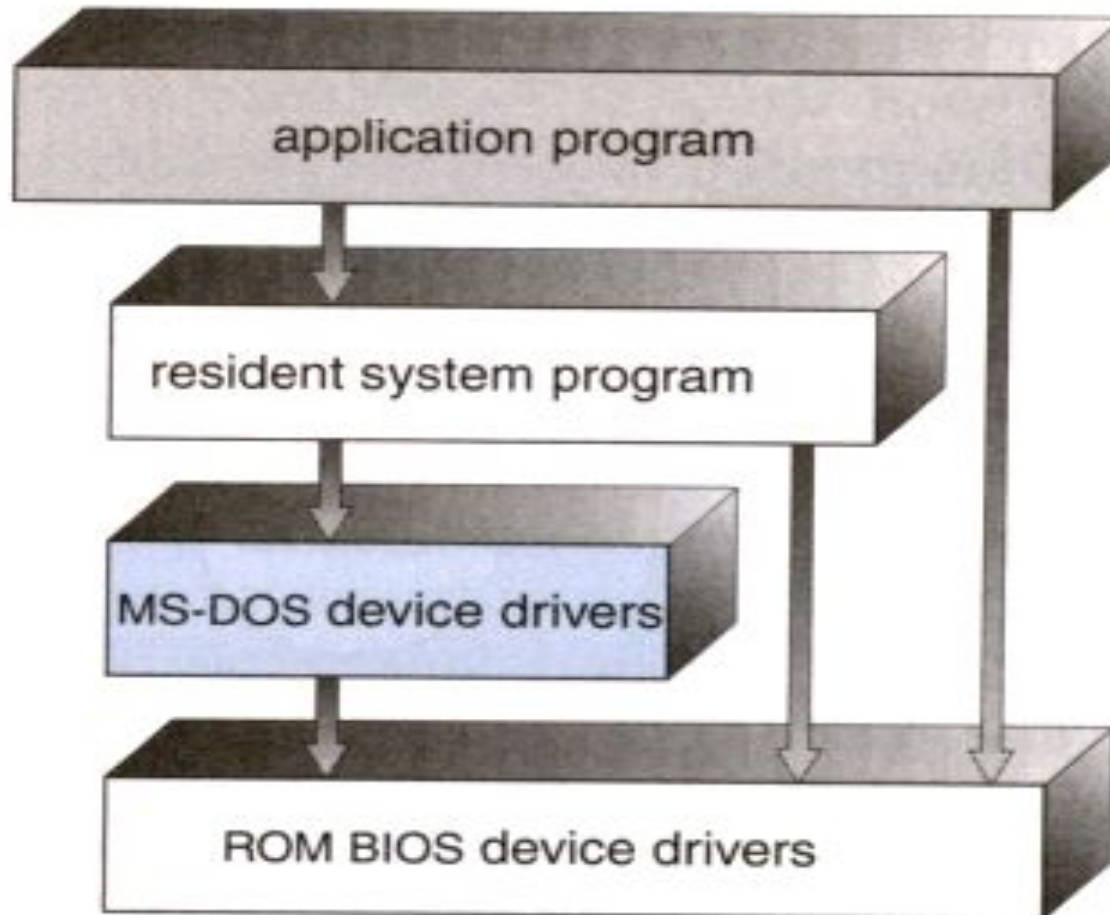
---



Monolithic Kernel

# MS-DOS Structure

---



# Operating System Design and Implementation (Cont.)

---

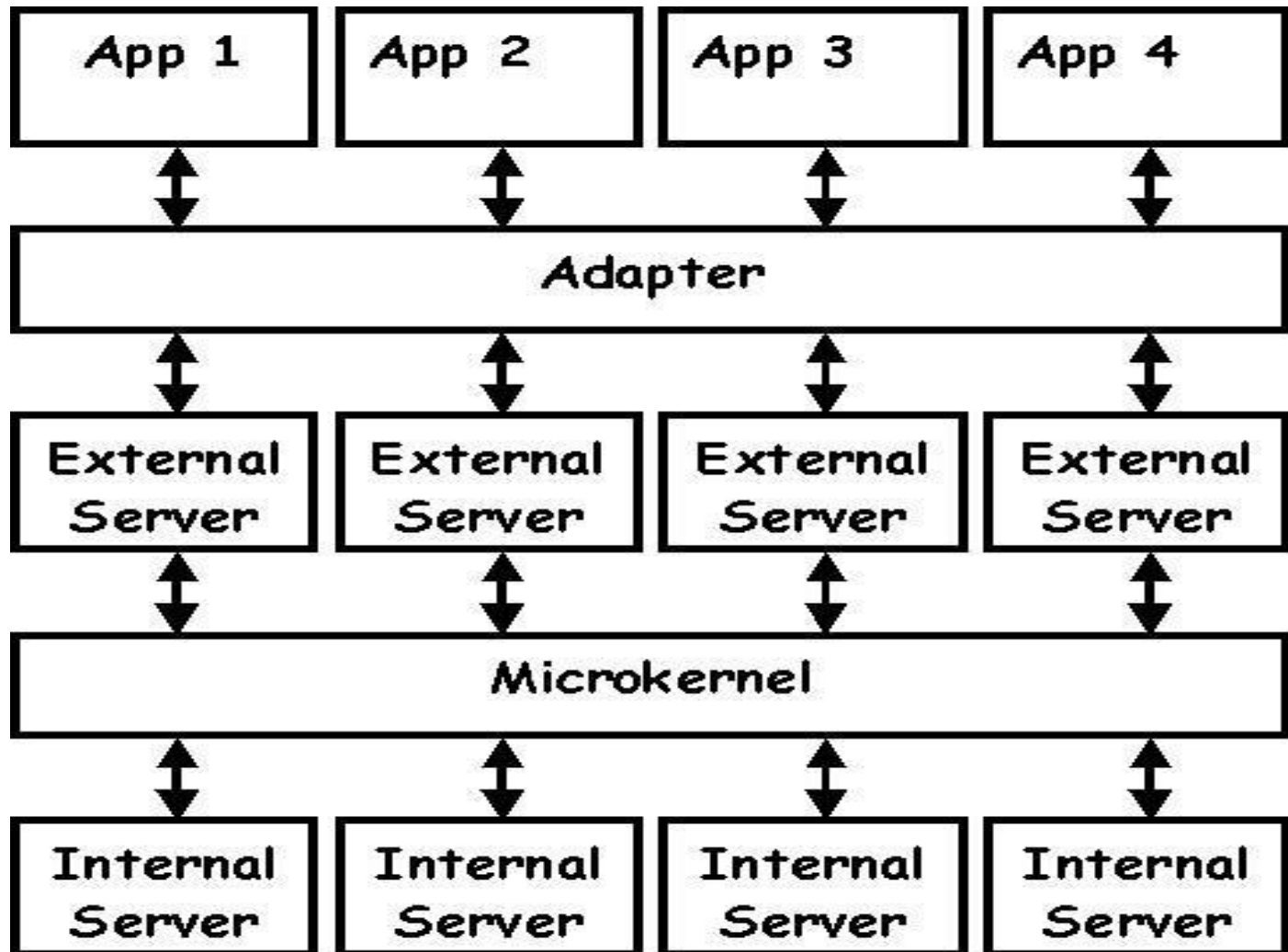
- Important principle to separate  
**Policy:** *What* will be done?  
**Mechanism:** *How* to do it?
- Mechanisms determine how to do something, policies decide what will be done
- The separation of policy from mechanism is a very important principle, it allows **maximum flexibility** if policy decisions are to be changed later (example – timer)
- Specifying and designing an OS is highly creative task of **software engineering**

# Microkernel Structure

---

- 2-10 bugs per thousand lines of code
  - Bugs in monolithic kernel having five million lines of code?
    - 4 Most of these not really fatal
- Services like filesystem access needs might be different for different applications – **need for customization**
- Achieve high reliability by splitting the operating system into small, well defined modules
  - Only one of these modules (**microkernel**) runs as kernel process
  - Other modules run as relatively powerless ordinary user processes
  - A bug in audio driver will at maximum cause the sound to stop
- Usually dominant in realtime, industrial, avionics, industrial applications and areas having high reliability requirements
  - Less used in desktop based systems
- **Mechanism** for doing things separated from **policy**, and **placed in kernel**
  - Simple abstractions, address space, IPC etc.

# Microkernel Structure



# Microkernel-Based OS Structure

(App1) (App2) ..... (Appn)  $\Rightarrow$  Each App in its own hardware address space

OS services

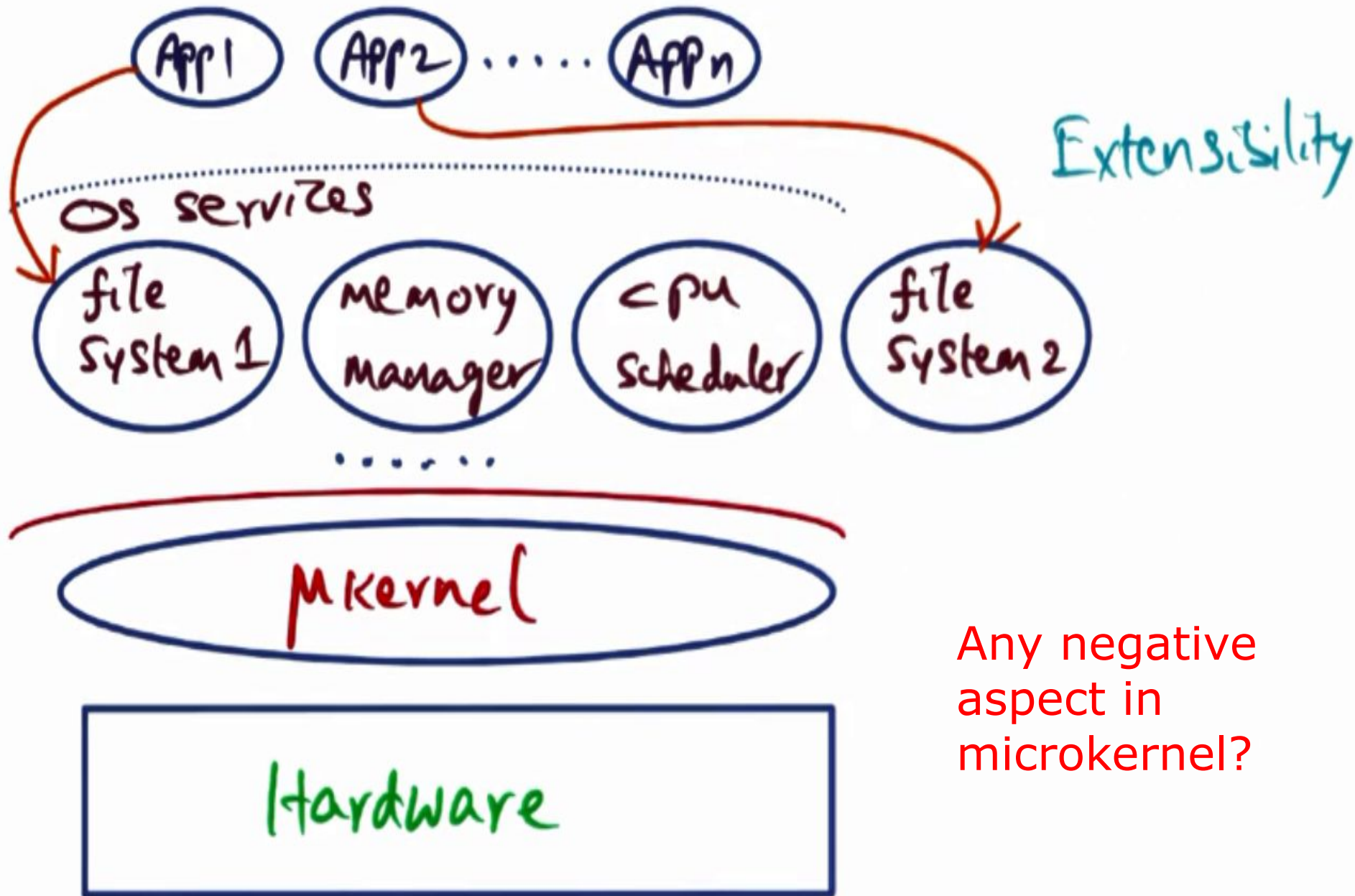
(file system) (memory manager) (cpu scheduler)  $\Rightarrow$  Each service in its own address space  
.....

mkernel  $\Rightarrow$  simple abstract  
- address space  
- ipc

Hardware

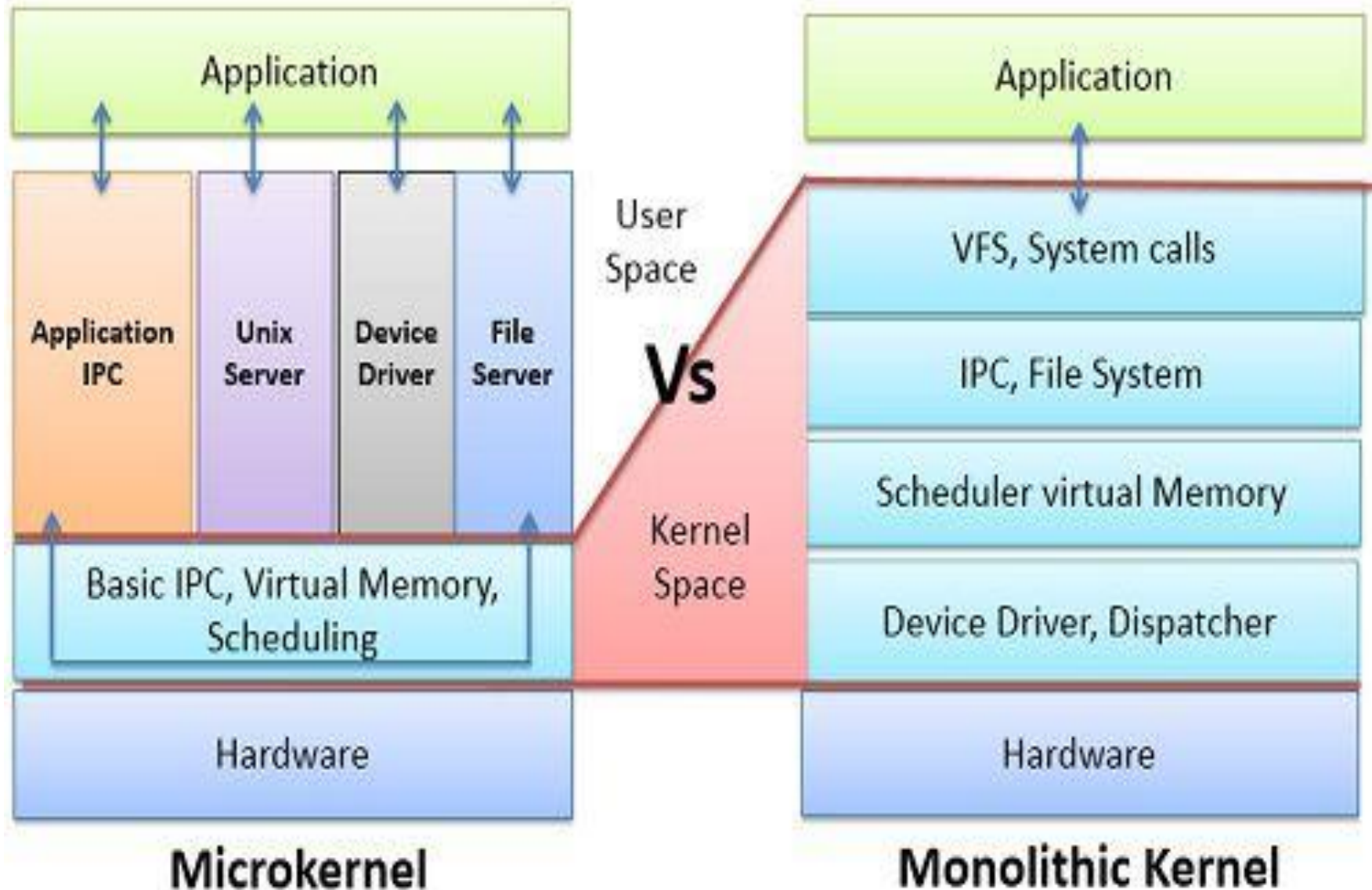


# Advantage of microkernel-based design



Any negative aspect in microkernel?

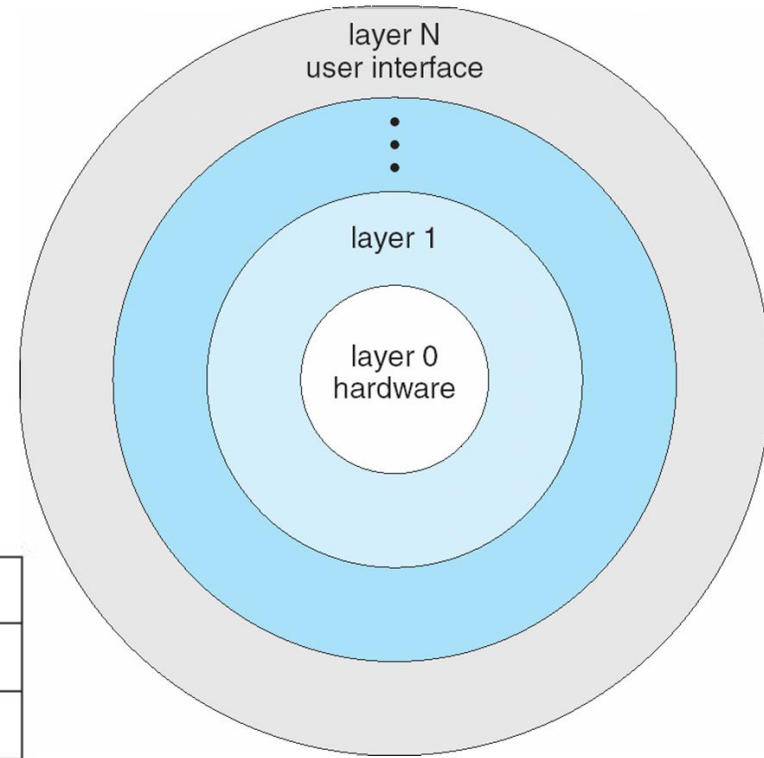
# Microkernel Structure



# Layered Approach

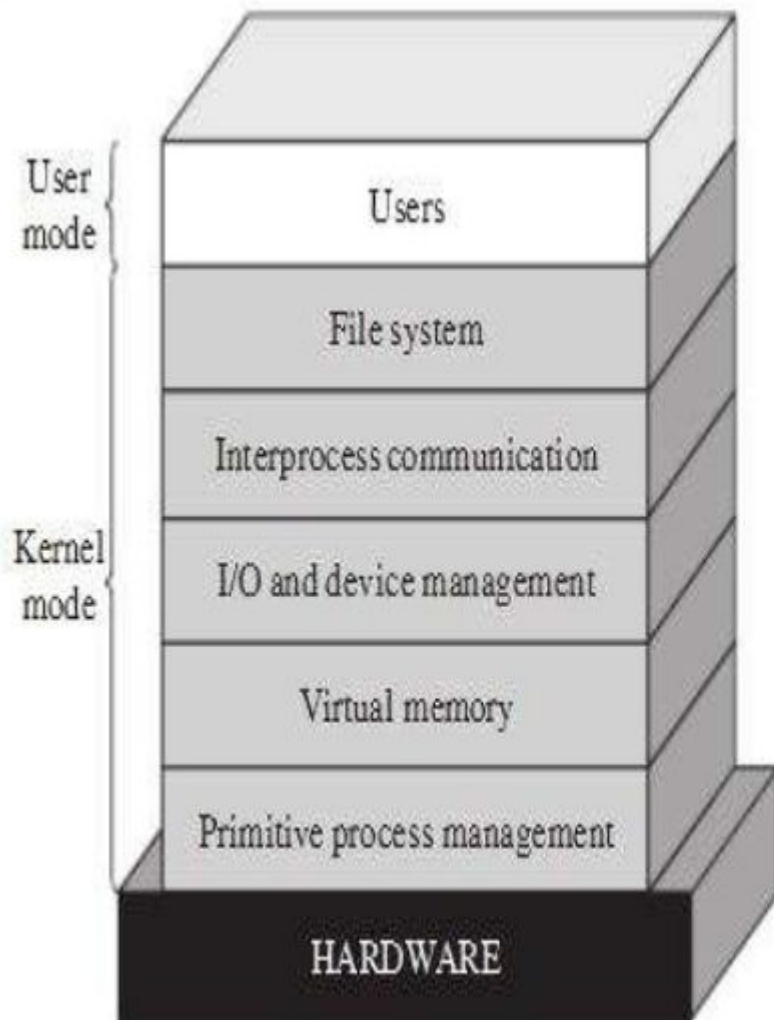
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

| Layer | Function                                  |
|-------|-------------------------------------------|
| 5     | The operator                              |
| 4     | User programs                             |
| 3     | Input/output management                   |
| 2     | Operator-process communication            |
| 1     | Memory and drum management                |
| 0     | Processor allocation and multiprogramming |

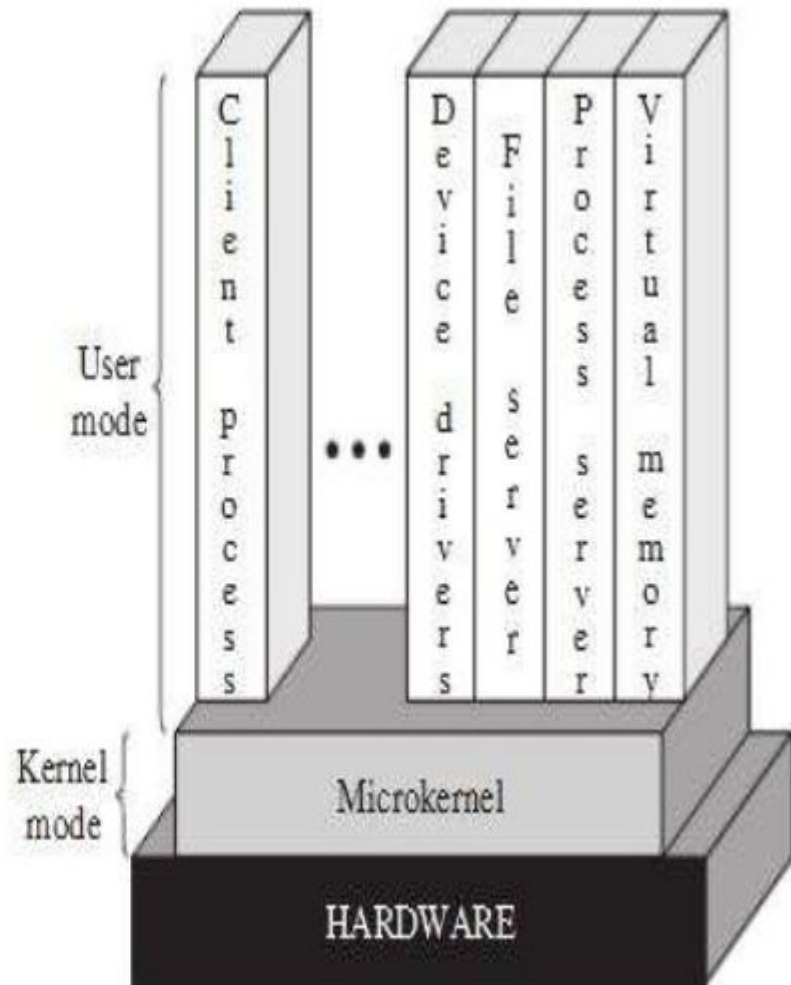


**Figure 1-25.** Structure of the THE operating system.

# Microkernel Structure



(a) Layered kernel



(b) Microkernel